
TECHNICAL LAB NOTE

OpenSoC : A RISC-V SoC from Silicon Setu, Brought Up on Crucible Workbench

Firmware, Lint, and RTL Simulation of the Level-1 SoC, End-to-End on **Crucible**

Crucible Platform Series

AT A GLANCE

Four Experiments, One Workspace

Four experiments take a real **RISC-V System-on-Chip** from source to verified waveform, all run through the **Crucible CLI (igny)**, all passing.

#	EXPERIMENT	TOOLS (VIA IGNY)	RESULT
1	Build RV32IM firmware	riscv-none-elf-gcc	✓ PASS
2	Static lint of the RTL	verilator	✓ PASS
3	GPIO simulation + waveform	verilator, gtkwave	✓ PASS
4	UART simulation + waveform	verilator, gtkwave	✓ PASS

CONTENTS

Table of Contents

01	Executive Summary	3
02	What is OpenSoC ? The Reference Design	4
03	The Crucible Workbench	5
04	Lab Setup	6
05	Experiment 1 : Build the Firmware	7
06	Experiment 2 : Static Lint of the RTL	8
07	Experiment 3 : GPIO Simulation and Waveform	9
08	Experiment 4 : UART Simulation and Waveform	10
09	Summary of Results and Conclusion	11

EXECUTIVE SUMMARY

Executive Summary

Bringing up a real **RISC-V System-on-Chip** normally means hand-assembling a toolchain: a cross-compiler to build firmware, a simulator to run the **RTL**, a waveform viewer to inspect it, and a build tool to tie the flow together, each installed separately, each version-matched by hand. That setup is fragile and rarely reproduces across machines.

This lab note demonstrates the complete front-end bring-up of the **OpenSoC Level-1 SoC**, an educational **RISC-V** chip design, on a managed workbench. Using **Crucible (igny)**, we stand up an isolated environment, install four catalogued tools, and run a full flow: cross-compile the firmware, lint the **RTL**, simulate two peripherals, and inspect the waveforms, all without installing a single tool by hand.

Four experiments, all passing, all run through **igny**:

#	EXPERIMENT	OUTCOME	OK
1	Build firmware	gpio_test_l1.hex, uart_test_l1.hex produced	✓
2	Lint the RTL	Clean Verilation report, no errors	✓
3	GPIO simulation	[TB] PASS, waveform captured	✓
4	UART simulation	[TB] PASS, waveform captured	✓

REPRODUCIBILITY

Because the toolset is locked with **igny** workspace lock, the entire lab is reproducible on any machine: *git clone ... && igny workspace sync* restores the identical toolchain, same compiler, same simulator, same versions, with no manual installation.

SECTION 2

What is OpenSoC?

OpenSoC is an educational **RISC-V System-on-Chip**: a small but complete computer on a chip, a CPU core, memory, a bus connecting them, and two peripherals (a **UART** serial port and **GPIO** pins). This note covers **Level 1**, the smallest configuration.

SYSTEM SPECIFICATION

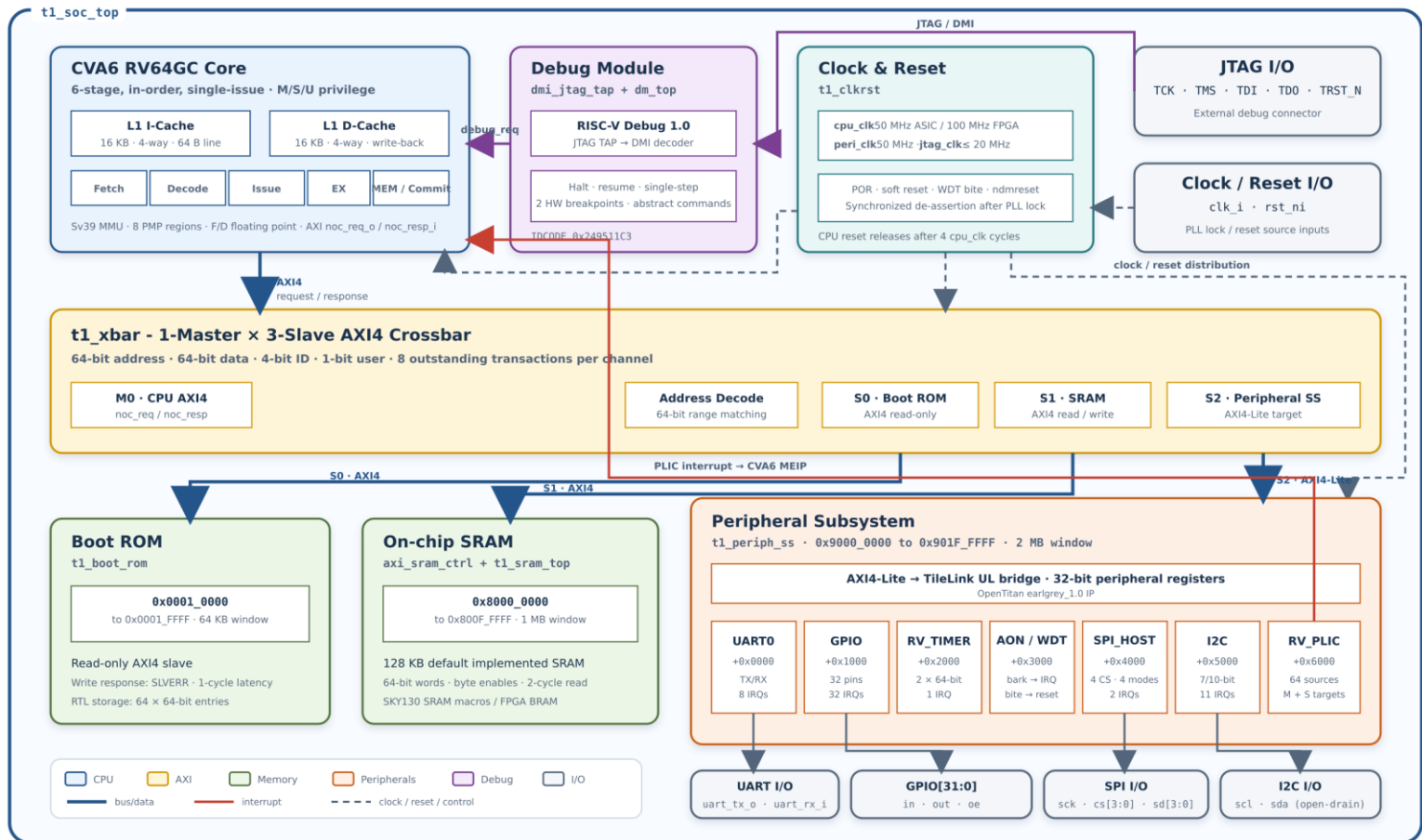
COMPONENT	DETAIL
CPU	Shakti E-class (RV32IM, 3-stage) . In sim, replaced by rv32i_iss, a behavioural ISS .
Bus	t1_bus_l1, 2-master x 2-slave 32-bit AXI4 interconnect
Memory	4 KB SRAM at 0x8000_0000; CPU boots directly from here
Peripherals	OpenTitan UART (0x9000_0000) + 16-bit GPIO (0x9000_1000) over TL-UL
Firmware	Bare-metal C, cross-compiled to a RISC-V hex image
Workbench	Crucible CLI (igny)

HOW SIMULATION WORKS

1. Compile, C firmware is cross-compiled for **RV32IM** and converted to a flat hex image the simulator loads.
2. Execute, the **ISS** (rv32i_iss) reads the hex and runs the **RISC-V** instructions cycle by cycle.
3. Drive the hardware, those instructions read/write the real **RTL**: bus, **SRAM**, **UART** and **GPIO**.
4. Model it fast, **Verilator** compiles the **RTL** into a fast C++ model; with **WAVES=1** every signal is dumped to .vcd.

OpenSoC Tier-1 - SoC Top-Level Architecture

Single-core CVA6 RV64GC · 1M × 3S AXI4 · On-chip SRAM · OpenTitan peripherals



Regenerated from OpenSoC Tier-1 Technical Specification v1.4 · Address windows follow the v1.1-corrected implemented map

LEVEL 1 vs LEVEL 2

Level 1 (this note) has **UART + GPIO**, 4 KB **SRAM**, direct-**SRAM** boot. **Level 2** adds SPI/I2C, larger memory, and a boot ROM. Wherever you see 'l1' in a filename it refers to this Level-1 config. Success is signalled via **TOHOST**: firmware writes 0x00000001 to 0x8000_OFF0 on pass.

SECTION 3

The Crucible Workbench

PLATFORM OVERVIEW

Ignition is a deep R&D company working at the intersection of software infrastructure and semiconductor engineering. **Crucible**, its environment-orchestration engine, versions tools and locks workspaces for reproducible workflows, so engineers focus on design not toolchains.

THE CRUCIBLE WORKBENCH

At the core is the **Crucible Workbench**, a managed-tooling environment that abstracts tool installation, versioning, and execution behind one clean interface. It binds specific tool versions to a named environment. Lock the toolset with **ign**y workspace lock; reproduce it with **ign**y workspace sync.

THE IGNY CLI

The **ign**y CLI is the primary interface to **Crucible**. Engineers use **ign**y env create to define an isolated environment, `ign y tool install` to populate it, and `ign y run <tool>` to execute without putting anything on the system PATH, eliminating environment drift and version conflicts.

TOOLCHAIN USED IN THIS LAB

ROLE	TOOL
Cross-compiler	riscv-none-elf-gcc 14.2.0
Simulator	Verilator 5.047
Waveform viewer	GTKWave 3.3.120
Build tool	make 4.4.1
Interface	ign y CLI

WHY VERILATOR?

OpenSoC can run on commercial simulators, but this note uses **Verilator**, a fully open-source simulator in the **Crucible** catalog, so every step here is reproducible on any machine with no license required.

SECTION 4

Lab Setup

Run this setup once. All four experiments reuse the same environment.

Step 1: Environment and Workspace

```
ign y env create silicon_sethu
ign y workspace create --path ./opensoc_soc1_l1 --env silicon_sethu
cd opensoc_soc1_l1
```

Step 2 : Install the Toolchain

```
ign y tool install riscv-none-elf-gcc
ign y tool install verilator
ign y tool install gtkwave
ign y tool install make
ign y workspace lock
```

Crucible Execution Output

```
C:\Users\ignyt\Desktop\demos >> ign y env create silicon_sethu
Environment 'silicon_sethu' created (stack=default)
C:\Users\ignyt\Desktop\demos >> ign y workspace create --path .\opensoc-soc1-l1\ --env silicon_sethu
Workspace 'opensoc-soc1-l1' created (env=silicon_sethu)
C:\Users\ignyt\Desktop\demos >> cd .\opensoc-soc1-l1\
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ign y compiler install riscv-none-elf-gcc
✓ Installed riscv-none-elf-gcc@14.2.0
Source: artifact
Path: C:\Users\ignyt\Crucible\tools\riscv-none-elf-gcc\14.2.0
Bound to env: silicon_sethu
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ign y tool install verilator
✓ Installed verilator@5.047.0
Source: artifact
Path: C:\Users\ignyt\Crucible\tools\verilator\5.047.0
Bound to env: silicon_sethu
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ign y tool install gtkwave
✓ Installed gtkwave@3.3.120
Source: artifact
Path: C:\Users\ignyt\Crucible\tools\gtkwave\3.3.120
Bound to env: silicon_sethu
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ign y tool install make
✓ Installed make@4.4.1
Source: artifact
Path: C:\Users\ignyt\Crucible\tools\make\4.4.1
Bound to env: silicon_sethu
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ign y workspace lock
Locked workspace 'opensoc-soc1-l1' (env='silicon_sethu') - 4 tool(s) -> crucible.lock
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> █
```

SECTION 4 (CONT.)

The Commands Every Experiment Uses

STEP	COMMAND	WHAT IT DOES
Build FW	<code>ign y run make -C sw <target></code>	Cross-compiles firmware into a .hex
Lint	<code>ign y run make verilator_lint_l1</code>	Static-checks the RTL , no sim
Simulate	<code>ign y run make verilator_<p>_l1</code>	Builds and runs the Verilator sim
Waveform	<code>... verilator_<p>_l1 WAVES=1</code>	Same, plus dumps a .vcd waveform
View	<code>ign y run gtkwave <path>/sim.vcd</code>	Opens the waveform in GTKWave

PASS CRITERIA

A simulation **PASSES** when its testbench prints [TB] **PASS** and the run exits cleanly. Under the hood, the firmware writes 0x00000001 to the **TOHOST** address on success; any other value is a decoded failure code.

Why **ign y**? **ign y** tool install downloads each tool once into a shared machine inventory and binds it to the `silicon_sethu` environment. A teammate reproduces the exact toolset with `git clone ... && ign y workspace sync`. Nothing is installed globally.

SECTION 5

Experiment 1: Build the Firmware

OBJECTIVE

Cross-compile the bare-metal C test programs into **RISC-V** hex images the simulator can load.

THEORY

The **SoC** runs no OS. Each test is a freestanding C program compiled for **RV32IM**, linked to run from **SRAM** at 0x8000_0000, then objcopy'd into a flat hex image (one 32-bit word per line) the **ISS** reads with \$readmemh. The build is driven by sw/Makefile, which locates the **Crucible RISC-V** compiler automatically.

PROCEDURE

```
ignyt run make -C sw gpio_l1
ignyt run make -C sw uart_l1
```

CRUCIBLE EXECUTION OUTPUT

```
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ignyt run make -- -C sw gpio_l1
make: Entering directory 'C:/Users/ignyt/Desktop/demos/opensoc-soc1-l1/sw'
C:/Users/ignyt/Crucible/tools/riscv-none-elf-gcc/14.2.0/riscv-none-elf-gcc/bin/riscv-none-elf-gcc -march=rv32im -mabi=ilp32 -nostdlib
-nostartfiles -ffreestanding -Os -g -T boot/eclass.ld boot/crt0.S tests/gpio_test.c -o tests/gpio_test_l1.elf
C:/Users/ignyt/Crucible/packages/ignyt_foundation_pkg_win_3.12.10.0/Python/python.exe ../scripts/bin2hex.py tests/gpio_test_l1.bin test
s/gpio_test_l1.hex
bin2hex: 492 bytes -> 123 words -> tests/gpio_test_l1.hex
[HEX] tests/gpio_test_l1.hex (123 words)
make: Leaving directory 'C:/Users/ignyt/Desktop/demos/opensoc-soc1-l1/sw'
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> ignyt run make -- -C sw uart_l1
make: Entering directory 'C:/Users/ignyt/Desktop/demos/opensoc-soc1-l1/sw'
C:/Users/ignyt/Crucible/tools/riscv-none-elf-gcc/14.2.0/riscv-none-elf-gcc/bin/riscv-none-elf-gcc -march=rv32im -mabi=ilp32 -nostdlib
-nostartfiles -ffreestanding -Os -g -T boot/eclass.ld boot/crt0.S tests/uart_test.c -o tests/uart_test_l1.elf
C:/Users/ignyt/Crucible/packages/ignyt_foundation_pkg_win_3.12.10.0/Python/python.exe ../scripts/bin2hex.py tests/uart_test_l1.bin test
s/uart_test_l1.hex
bin2hex: 724 bytes -> 181 words -> tests/uart_test_l1.hex
[HEX] tests/uart_test_l1.hex (181 words)
make: Leaving directory 'C:/Users/ignyt/Desktop/demos/opensoc-soc1-l1/sw'
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> |
```

✓ **EXPERIMENT 1 : PASS** · gpio_test_l1.hex (123 words), uart_test_l1.hex (181 words) built via riscv-none-elf-gcc 14.2.0

SECTION 6

Experiment 2 : Static Lint of the RTL

OBJECTIVE

Run **Verilator**'s static analysis over the entire Level-1 RTL before simulating, to catch structural issues cheaply.

THEORY

Linting elaborates the design and checks it without running a simulation , fast feedback on width mismatches, undriven/multiply-driven signals, latches, and ambiguous tops. Because several modules could each be a top, the lint pins the true top with --top-module t1_soc_top_eclass.

PROCEDURE

```
igny run make verilator_lint_l1
```

CRUCIBLE EXECUTION OUTPUT

```
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1 >> cd .\CRUCIBLE\sources\  
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1\crucible_folder >> igny run make verilator_lint_l1  
=== Verilator lint: L1 RTL ===  
- V e r i l a t i o n   R e p o r t: Verilator 5.047 devel rev v5.046-232-g9f4546fcb (mod)  
- Verilator: Built from 1.611 MB sources in 25 modules, into 0.274 MB in 3 C++ files needing 0.000 MB  
- Verilator: Walltime 0.090 s (elab=0.011, cvt=0.033, bld=0.000); cpu 0.000 s on 1 threads; allocated 22.695 MB  
  Log: build/veri_lint_l1.log  
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1\crucible_folder >> |
```

✓ EXPERIMENT 2 : PASS · 25 modules elaborated, no warnings or errors, exit 0

SECTION 8

Experiment 4 : UART Simulation and Waveform

OBJECTIVE

Run firmware that transmits over the **UART**; confirm correct serial framing and data in the log and waveform.

THEORY

A **UART** sends a byte as a frame: start bit, 8 data bits, stop bit (**8N1**), at a rate set by a baud divisor. The firmware configures the **UART** and transmits bytes; the testbench loops each frame back so the firmware reads and verifies them. Here the baud gives 8 cycles/bit, so one frame is 80 cycles plus echo.

```
#define UART_CTRL (*(volatile uint32_t*)(UART_BASE+0x10u))
#define UART_WDATA (*(volatile uint32_t*)(UART_BASE+0x1Cu))
UART_CTRL = 0x2000u | (1u<<16) | (1u<<17); // NCO + TX_EN + RX_EN
UART_WDATA = 0x55u; // transmit; testbench echoes it back
```

PROCEDURE

```
ign run make verilator_uart_l1 (OR)
ign run make verilator_uart_l1 WAVES=1
ign run gtkwave ..\build\verilator\uart_l1\sim.vcd
```

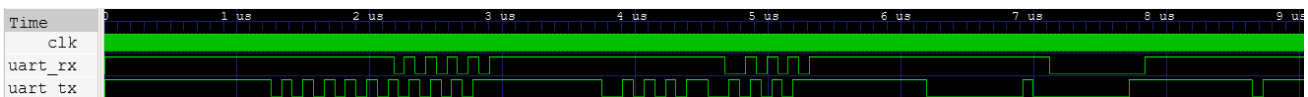
CRUCIBLE EXECUTION OUTPUT

```
C:\Users\ignyt\Desktop\demos\opensoc-soc1-l1\crucible_folder >> ign run make verilator_uart_l1 WAVES=1

=== Verilator uart_l1: compile ===
- Verilator Report: Verilator 5.047 devel rev v5.046-232-g9f4546fcb (mod)
- Verilator: Built from 1.776 MB sources in 26 modules, into 0.660 MB in 10 C++ files needing 0.001 MB
- Verilator: Walltime 0.158 s (elab=0.006, cvt=0.083, bld=0.000); cpu 0.000 s on 1 threads; allocated 25.246 MB
[TB] =====
[TB] PASS -- all checks passed [LEVEL-1 (Quick)]
[TB] =====
- crucible_folder/verif/tb/t1_eiclass_uart_tb.sv:201: Verilog $finish
- Simulation Report: Verilator 5.047 devel
- Verilator: $finish at 11us; walltime 0.008 s; speed 0.000 s/s
- Verilator: cpu 0.000 s on 1 threads; allocated 5 MB
[WAVE] build/verilator/uart_l1/sim.vcd

[TB] PASS -- all checks passed [LEVEL-1 (Quick)]
=== uart_l1: PASS ===
```

WAVEFORMS



✓ **EXPERIMENT 4 : PASS** · TOHOST 0x00000001 @ 1,111 cycles; waveform captured

SECTION 9

Summary of Results

#	EXPERIMENT	CATEGORY	RESULT
1	Build firmware	Cross-compile	✓ PASS
2	Verilator lint	Static analysis	✓ PASS
3	GPIO simulation	Functional sim	✓ PASS
4	UART simulation	Functional sim	✓ PASS

This lab note took a real **RISC-V System-on-Chip** through the core front-end steps of a chip-design flow , firmware → lint → simulation → waveform inspection , and ran the entire thing through a single **Crucible** workspace. No tool was installed by hand. The cross-compiler, simulator, waveform viewer, and build tool were all provisioned, version-pinned, and executed through **ignpy**.

Reproducible on any machine:

```
git clone <repo>
cd <repo>
ignpy workspace sync          # restores the identical, locked toolchain
```

FULL COMMAND SEQUENCE

```
# workspace setup
ignpy env create silicon_sethu
ignpy workspace create --path ./silicon-sethu --env silicon_sethu
cd silicon-sethu
ignpy tool install riscv-none-elf-gcc verilator gtkwave make
ignpy workspace lock

# exp 1 : firmware | # exp 2 : lint
ignpy run make -C sw gpio_l1 uart_l1
ignpy run make verilator_lint_l1

# exp 3 & 4 : simulate + waveform
cd .\crucible_folder
ignpy run make verilator_gpio_l1 WAVES=1
ignpy run gtkwave ..\build\verilator\gpio_l1\sim.vcd
ignpy run make verilator_uart_l1 WAVES=1
ignpy run gtkwave ..\build\verilator\gpio_l1\sim.vcd
```